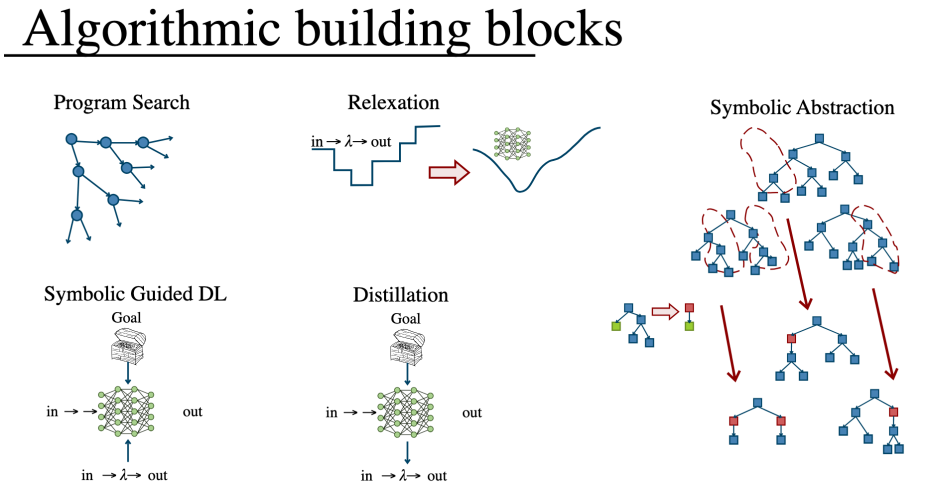
LECTURE 16

# Neurosymbolic programming

Earlier in the course, we contrasted machine learning—where the goal is to learn a function from data— with inductive program synthesis, where we are also interested in discovering a function that fits data, but where we want the function to be expressed as a program in a given programming language, and where we may have structural or additional behavioral constraints that the program needs to satisfy. More recently, in this unit, we saw how deep neural networks in general—and large language models in particular— can be used to solve program synthesis problems. But even though deep learning is an essential component in these approaches, the final goal is still to find a program that meets the desired behavioral and structural constraints. For this lecture, we explore a middle point between machine learning and program synthesis that we term **Neurosymbolic Programming (NSP)**. Similar to program synthesis, the goal is to discover a function from data that meets certain structural and behavioral constraints, but we are not fully restricted to the programs that are expressible in a given programming language. The programs can be hybrids between traditional code and neural networks, and in some cases they can be compositions of neural networks with constraints that give them some of the modularity of programs. In short, the goal is to *learn models that capture symbolic knowledge in the form of program structures*.




In order to do this, we are going to be relying on a set of algorithmic build-

ing blocks that will allow us to combine the benefits of deep learning with the benefits of symbolic program synthesis.

- **Program search.** The ability to search for programs that satisfy a set of examples using the techniques from the past two units is a key building block in NSP
- **Relaxation.** This is the ability to take a traditional symbolic program and make it differentiable. This can be done either by symbolically "smoothing" the program element by element [1] or by training a neural network to approximate the behavior of the program.
- **Symbolic guided deep learning.** This is the ability to train a neural network while constraining the program to behave like a program written in a symbolic programming language.
- **Distillation.** This is the ability to extract a symbolic program using the neural network as a teacher.
- **Symbolic abstraction (a.k.a Component discovery).** This corresponds to the component discovery techniques that we introduced in Lecture 9.

Program search and symbolic abstraction have already been covered in this course, so for the rest of the lecture, we focus on relaxation, distillation and symbolic guided deep learning.

## 16.1 Relaxation

Symbolic vs. Neural

Symbolic relaxation == program smoothing

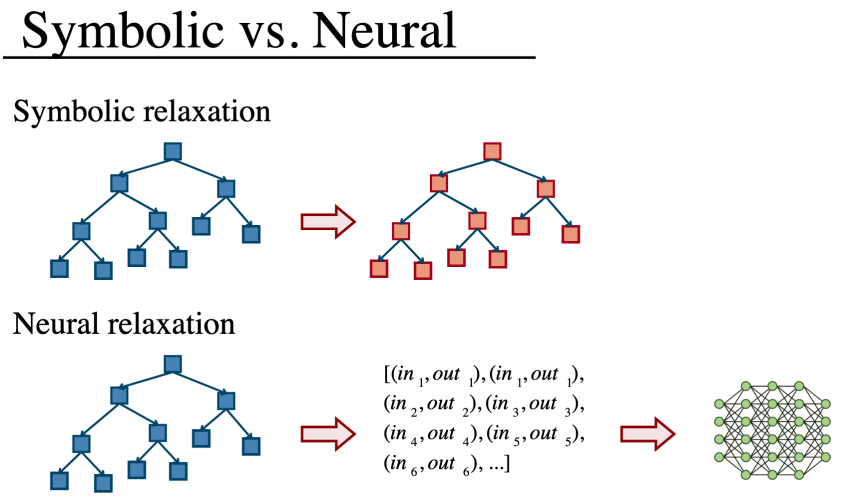
Symbolically replace individual language constructs with hand-crafted continuous approximations

Smoothed program preserves symbolic structure

Neural relaxation == Neural surrogates

Use the program to produce data to train a neural network to emulate its behavior

Program structure can still be exploited symbolically to improve surrogate quality (see work by Alex Renda)




The idea of relaxation is to take a symbolic program and make it differentiable. There are two basic appraoches to do this. The first approach is to do it symbolically, by replacing each program element with a differentiable approximation. Swarat Chaudhuri and I wrote an early paper showing how to do this [1]. The goal in that paper was to derive a smooth approximation of a program that could be used to optimize the parameters of the program through numerical optimization. More recently, there has been significant interest in the area of **differentiable programming**, which provides differentiable analogs to common programming constructs which can be composed to form differentiable programs [2].

An alternative approach is to use the program as a data generator to train a neural network that imitates the behavior of the program. The basic idea is simple, one can generate random inputs for a program and then train a neural network to predict the outputs from the inputs. The resulting neural network can then be used as a differentiable approximation of the program.

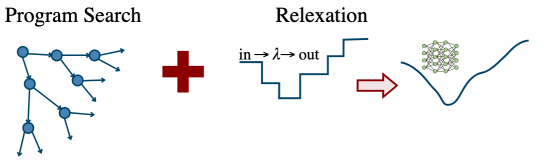
Alex Renda, Yi Ding and and Michael Carbin together explored some applications of these *Neural Surrogates* [3]. The most straightforward application is to use neural surrogates to optimize program parameters through gradient descent, but they also highlight other applications such as speeding up the execution of an otherwise slow algorithm. In a different paper, the same authors also show how to better leverage program structure in order to get samples that better represent the different behaviors of the program [4].

## 16.2 Combining Relaxation and Program search

Relaxation can be combined with program search both to find programs that match a set of input-output examples, but also to find programs that combine

symbolic and neural elements. This was demonstrated by Shah et al. in their work on the NEAR algorithm [5].

The algorithm builds on the idea of *angelic non-determinism*, which had previously been proposed as a way to determine whether a partial program could possibly be completed to satisfy a given specification [6]. The idea is that given a partial program, completing the program with pieces of code that work for all inputs may be quite hard, but imagine if we could execute the partial program such that every time execution reaches an uknown piece of code, we ask an oracle to produce a value that allows the program satisfy the specification. The job of the oracle is simpler than the overall synthesis problem, because it does not have to figure out a program that works for all inputs, only a value that works for the current input. If no such value can be found, then clearly there is no way the partial program could be completed to satisfy the specification for all inputs.

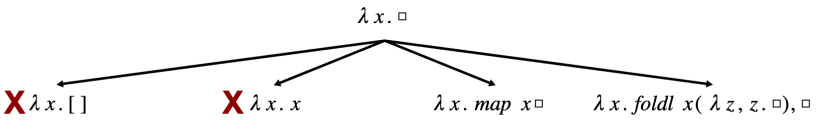



NEAR

Learning Differentiable Programs with Admissible Neural Heuristics,
Ameesh Shah, Eric Zhan, Jennifer J. Sun, Abhinav Verma, Yisong Yue,
Swarat Chaudhuri

## Key ideas

Start with type directed top-down search




Relax a candidate program filling any holes with neural networks.




Training loss becomes an *admissible heuristic* to guide the search

# Admissible heuristics

A heuristic is said to be *admissible* if it never overestimates the cost to reach the goal

Cost of a completed program $p$: $s(p) + \text{Loss}(p)$

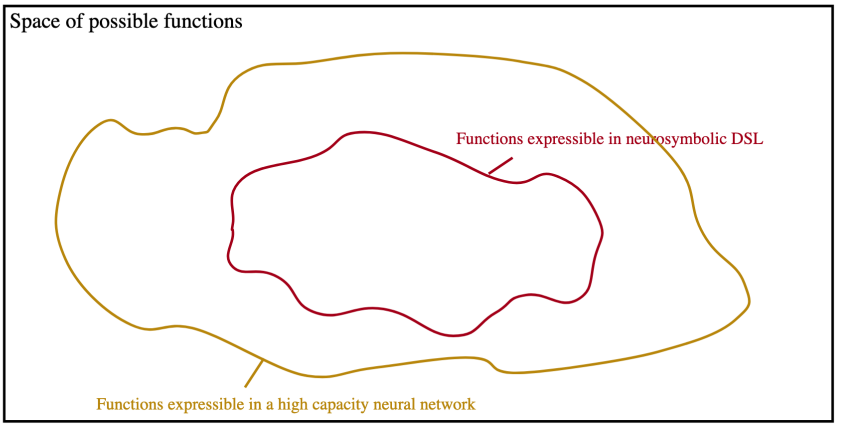
structural cost — training loss

Cost of a partial program $p^*$: $s(p^*) + \text{Loss}(\text{Relaxed}(p^*)) = s(p^*) + d(p^*)$

Under some assumptions, if $p \in \text{Completions}(p^*)$ then

$s(p) + \text{Loss}(p) > s(p^*) + d(p^*)$

# Expressive power



## A* Search

Priority queue of current leaf nodes:

Sorted by $s(P^*) + d(P^*)$

Pop off top program $P^*$

If $P^*$ is complete, terminate

Else, expand $P^*$, add child nodes to priority queue

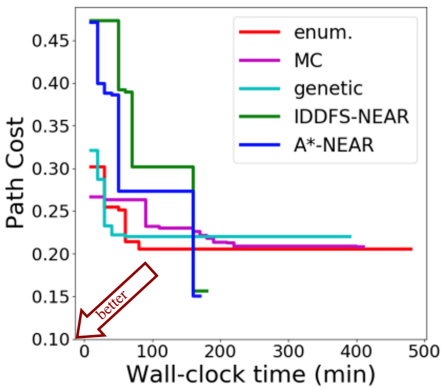
**Guarantee** : if $d(P^*)$ is admissible, A* will return optimal $P$

·Tighter $d(P^*)$ prunes more aggressively

·Uninformed $d(P^*)$ (e.g., always 0) => uninformed search

## Results

NEAR is more than an order of magnitude better than the baselines.




NEAR builds on this idea by training a neural network to serve as the oracle. The algorithm works like a traditional top-down program search, but every time it generates a new partial program as part of the search, it trains a neural network to serve as the oracle for each hole in the partial program. If no such oracle can be trained, the assumption is that the partial program cannot be completed to satisfy the specification, so the search choses a different branch to explore.

If we assume that neural networks can approximate any function representable by a program fragment in the language, and if we assume we are training the neural network to convergence, then the error made by the hybrid neural-symbolic program is a lower bound on the error made by any completion of the partial program. This allows us to use the error of the neural-

symbolic hybrid program as an admissible heuristic for a graph search over the space of partial programs.

The algorithm has an added advantage, which is that it is an anytime algorighm. The process can be stopped at any time, and the resulting neurosymbolic program will have some neural components in the middle of a symbolic structure.

## 16.3 Distillation

In 2018, Verma et al. introduced the idea of extracting a symbolic program from a trained neural network [7] as part of their work on *Programmatically Interpretable Reinforcement Learning*. The basic idea was to train a neural network to solve a reinforcement learning task and then extracting a symbolic program that approximates the behavior of the neural network. We did something similar in a 2018 paper led by Osbert Bastani [8], where the goal was to learn a symbolic representation of a neural controller. In both cases, the goal was to turn a neural network into a symbolic representation, but why?

In the case of Bastani et al. the goal was verification. While there has been some progress directly verifying properties of neural networks, a small program or a small decision tree are much easier to verify than a neural network. But Verma et al. demonstrated that there could be other advantages too. By carefully selecting the space of programs for the distilled function, it is possible to enforce certain properties in the generated program that would have been more difficult to enforce in the neural network.

For example, Verma et al. focused on a control problem, specifically the problem of driving a car in a racing simulator. For the distilled representation, they chose a space of switched PID controllers, which correspond to a class of controllers that are widely used in control systems. By distilling the neural network into a switched PID controller, they are able to capture the knwoledge that the neural network acquired during RL for how to drive the car, but in a controller that proved to be more robust, in addition to being more interpretable.

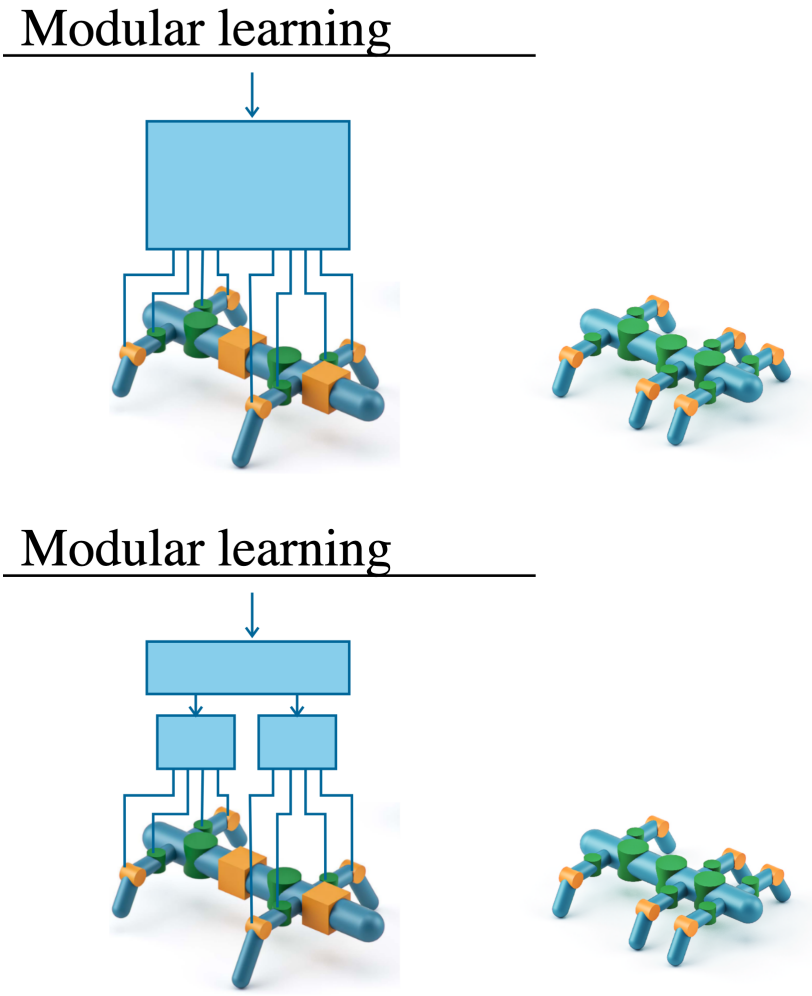
The simplest approach to distillation is to simply use the neural network as a teacher to generate input-output examples that can be used to drive an off-the-shelf inductive program synthesis algorithm. In this sense, the process is the exact inverse of the neural relaxation approach described earlier, where a program is used to generate data to train a neural network.

Bastani et al. had an additional insight that not all examples were equally important [8]. In particular, if the neural policy was trained through reinforcement learning, it is possible to use the Q function to identify states where many different actions produce similar rewards, and states where only a few actions produce high rewards. By focusing the distillation process on the states where

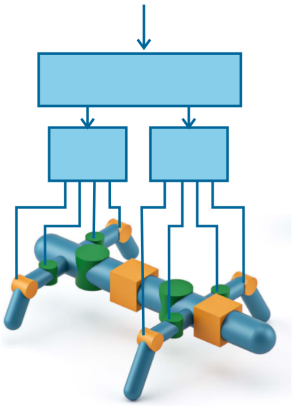
the choice of action matters the most, it is possible to generate more concise symbolic representations that don't try unnecessarily to capture complex behaviors that don't matter for the overall performance of the policy.

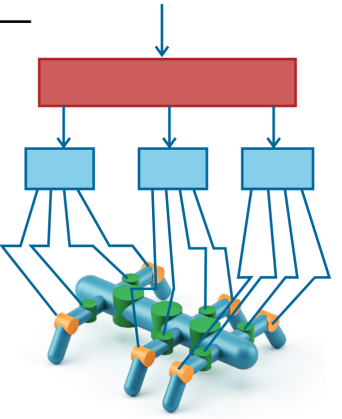
With a proper choice of DSL, it is possible to capture neural networks of significant complexity. For example, in a 2022 paper, we used distillation to extract a symbolic representation of a policy that was used to coordinate a team of quadcopters [9].

### 16.4 Symbolic guided deep learning

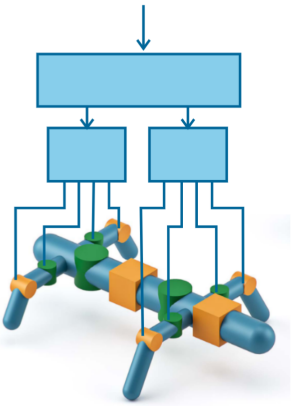


## Modular learning





## Modular learning



Two failure modes

Global controller does too much

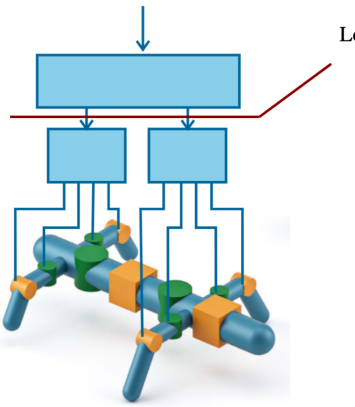
No benefit to having the modules

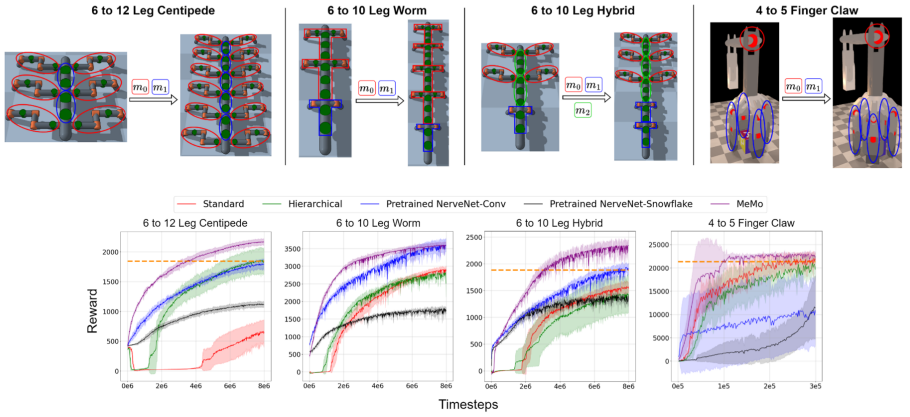
Global controller does too little

Modules become overspecialized

How to enforce a good partition?

## Modular learning






In neural relaxation, we already have a concrete program, and the goal is to find a neural network that behaves like that program. A more general version of this is when we don't have a program, but we have a set of structural constraints, and we want the neural network to behave like some program satisfying those structural constraints. One example of this is our paper on MeMo [10]. In this paper, the goal is to train modular controllers to control a modular robot constructed by joining together a set of actuated segments. The baseline for this is to simply train a neural network to control every actuator in the robot, but this approach does not leverage the modular structure of the robot, and it does not generalize well to new robot configurations.

The goal of MeMo is to train a modular controller where each sub-assembly in the robot has its own controller, and there is a global coordinator that learns to coordinate the different sub-controllers to control the overall robot. In other words, we have in mind a program to control this robot, where on each iteration, a global coordinator takes a global state as input and produces a set of sub-goals for each sub-controller, and then each sub-controller takes the sub-goal and the local state as input and produces an action for its corresponding sub-assembly.

If we had such a concrete program, we could use the neural relaxation techniques described earlier to train a neural network for each individual module. But we don't have such a program, we only have the high level structure. We can structure the neural network to have the same structure as the program we have in mind, but the challenge is to get a proper division of labor between the different modules. How do we prevent the global coordinator from taking over all the work and just learning a monolithic policy that controls the whole robot?

The key idea in MeMo is to use noise injection to create a bottleneck that prevents the global coordinator from taking over all the work. By injecting noise

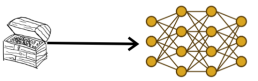
in the communication between the global coordinator and the sub-controllers, we force the global coordinator to learn to produce sub-goals that are robust to noise.

This is just one example of how to use symbolic constraints to guide the training of a neural network.

## 16.5 Adaptive teaching

# Idea 1: Adaptive teaching

Iterate between Distillation and Symbolic Guided DL to reach a function that is representable in the DSL

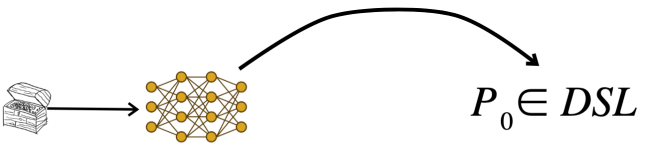


1. Start by training a network that satisfies the goal.

Jeevana Priya Inala, Osbert Bastani, Zenna Tavares, Armando Solar-Lezama: Synthesizing Programmatic Policies that Inductively Generalize, ICLR 2020

# Idea 1: Adaptive teaching

Iterate between Distillation and Symbolic Guided DL to reach a function that is representable in the DSL

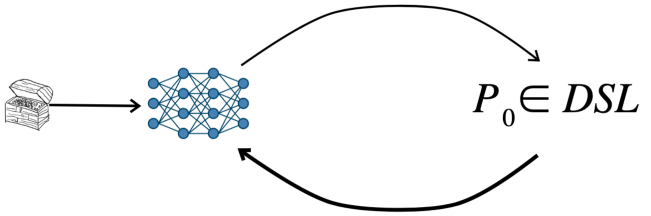



2. Distill the best program that matches the network, even if it is not very good.

Jeevana Priya Inala, Osbert Bastani, Zenna Tavares, Armando Solar-Lezama: Synthesizing Programmatic Policies that Inductively Generalize, ICLR 2020

# Idea 1: Adaptive teaching

Iterate between Distillation and Symbolic Guided DL to reach a function that is representable in the DSL

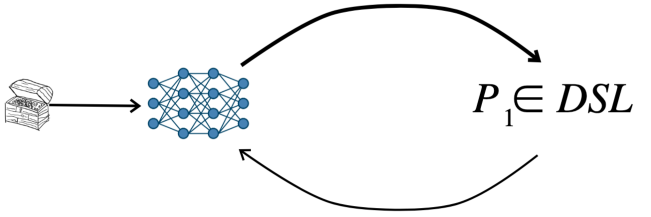



3. Retrain the network with an additional objective that its behavior should attempt to match the program.

Jeevana Priya Inala, Osbert Bastani, Zenna Tavares, Armando Solar-Lezama: Synthesizing Programmatic Policies that Inductively Generalize, ICLR 2020

# Idea 1: Adaptive teaching

Iterate between Distillation and Symbolic Guided DL to reach a function that is representable in the DSL

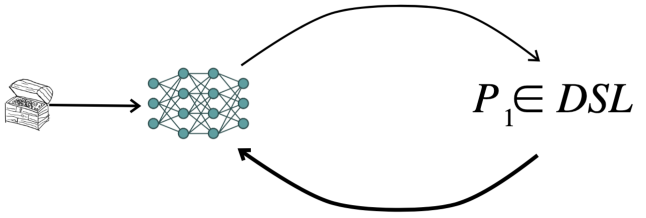



4. Repeat the cycle as many times as necessary.

Jeevana Priya Inala, Osbert Bastani, Zenna Tavares, Armando Solar-Lezama: Synthesizing Programmatic Policies that Inductively Generalize, ICLR 2020

# Idea 1: Adaptive teaching

Iterate between Distillation and Symbolic Guided DL to reach a function that is representable in the DSL

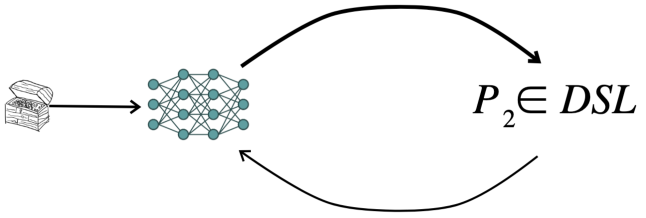



4. Repeat the cycle as many times as necessary.

Jeevana Priya Inala, Osbert Bastani, Zenna Tavares, Armando Solar-Lezama: Synthesizing Programmatic Policies that Inductively Generalize, ICLR 2020

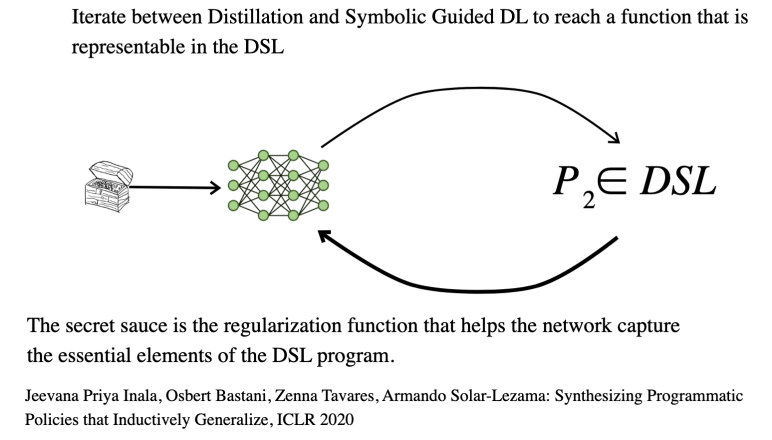
# Idea 1: Adaptive teaching

Iterate between Distillation and Symbolic Guided DL to reach a function that is representable in the DSL




4. Repeat the cycle as many times as necessary.

Jeevana Priya Inala, Osbert Bastani, Zenna Tavares, Armando Solar-Lezama: Synthesizing Programmatic Policies that Inductively Generalize, ICLR 2020




A lot of benefits can be had by combining the different building blocks we have described. For example, one challenge with distillation is that the neural network may have learned behaviors that are not easily captured in the symbolic representation, and this can lead to a large error in the distilled program. By using the symbolic guided deep learning, it is possible to train the neural network to behave in a way that is more easily captured by the symbolic representation, which can lead to better distillation results. This is the main idea behind *Adaptive teaching*, a technique we introduced in a 2022 paper [11].

Adaptive teaching iterates between training *teacher* to solve a task, and then training a *student* to imitate the teacher, but with a symbolic representation. In the original paper, the teacher was a heavily parameterized model that was trained to solve a specific task instance, and the student was a hybrid automata that combined symbolic finite state machine structure with a small parametric model to control the behavior at each mode of the state machine. The goal was to train the student to imitate the trajectories produced by the teacher. What made the teaching adaptive was that if the student was unable to faithfully imitate the teacher, the teacher would be retrained with an additional loss term that encouraged it to produce trajectories that were easier for the student to imitate.

# References


[1] Swarat Chaudhuri and Armando Solar-Lezama. Smooth interpretation. In Benjamin G. Zorn and Alex Aiken, editors, *Proceedings of the 2010 ACM SIGPLAN Conference on Programming Language Design and Implementation, PLDI 2010, Toronto, Ontario, Canada, June 5-10, 2010*, pages 279–291. ACM, 2010. doi: 10.1145/1806596.1806629. URL https://doi.org/10.1145/1806596.1806629.

[2] Martín Abadi and Gordon D. Plotkin. A simple differentiable programming language. *Proc. ACM Program. Lang.*, 4(POPL):38:1–38:28, 2020. doi: 10.1145/3371106. URL https://doi.org/10.1145/3371106.

[3] Alex Renda, Yi Ding, and Michael Carbin. Programming with neural surrogates of programs. In Wolfgang De Meuter and Elisa L. A. Baniassad, editors, *Onward! 2021: Proceedings of the 2021 ACM SIGPLAN International Symposium on New Ideas, New Paradigms, and Reflections on Programming and Software, Virtual Event / Chicago, IL, USA, October 20-22, 2021*, pages 18–38. ACM, 2021. doi: 10.1145/3486607.3486748. URL https://doi.org/10.1145/3486607.3486748.

[4] Alex Renda, Yi Ding, and Michael Carbin. Turaco: Complexity-guided data sampling for training neural surrogates of programs. *Proc. ACM Program. Lang.*, 7(OOPSLA2):1648–1676, 2023. doi: 10.1145/3622856. URL https://doi.org/10.1145/3622856.

[5] Ameesh Shah, Eric Zhan, Jennifer Sun, Abhinav Verma, Yisong Yue, and Swarat Chaudhuri. Learning differentiable programs with admissible neural heuristics. In H. Larochelle, M. Ranzato, R. Hadsell, M.F. Balcan, and H. Lin, editors, *Advances in Neural Information Processing Systems*, volume 33, pages 4940–4952. Curran Associates, Inc., 2020. URL https://proceedings.neurips.cc/paper_files/paper/2020/file/342285bb2a8cadef22f667eeb6a63732-Paper.pdf.

[6] Rastislav Bodík, Satish Chandra, Joel Galenson, Doug Kimelman, Nicholas Tung, Shaon Barman, and Casey Rodarmor. Programming with angelic nondeterminism. In Manuel V. Hermenegildo and Jens Palsberg, editors, *Proceedings of the 37th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages, POPL 2010, Madrid, Spain, January 17-23, 2010*, pages 339–352. ACM, 2010. doi: 10.1145/1706299.1706339. URL https://doi.org/10.1145/1706299.1706339.

[7] Abhinav Verma, Vijayaraghavan Murali, Rishabh Singh, Pushmeet Kohli, and Swarat Chaudhuri. Programmatically interpretable reinforcement learning. In Jennifer Dy and Andreas Krause, editors, *Proceedings of the 35th International Conference on Machine Learning*, volume 80 of *Proceedings of Machine Learning Research*, pages 5045–5054. PMLR, 10–15 Jul 2018. URL https://proceedings.mlr.press/v80/verma18a.html.

[8] Osbert Bastani, Yewen Pu, and Armando Solar-Lezama. Verifiable reinforcement learning via policy extraction. In Samy Bengio, Hanna M. Wallach, Hugo Larochelle, Kristen Grauman, Nicolò Cesa-Bianchi, and Roman Garnett, editors, *Advances in Neural Information Processing Systems 31: Annual Conference on Neural Information Processing Systems 2018, NeurIPS 2018, December 3-8, 2018, Montréal, Canada*, pages 2499–2509, 2018. URL https://proceedings.neurips.cc/paper/2018/hash/e6d8545daa42d5ced125a4bf747b3688-Abstract.html.

[9] Jeevana Priya Inala, Yichen Yang, James Paulos, Yewen Pu, Osbert Bastani, Vijay Kumar, Martin C. Rinard, and Armando Solar-Lezama. Neurosymbolic transformers for multi-agent communication. In Hugo Larochelle, Marc'Aurelio Ranzato, Raia Hadsell, Maria-Florina Balcan, and Hsuan-Tien Lin, editors, *Advances in Neural Information Processing Systems 33: Annual Conference on Neural Information Processing Systems 2020, NeurIPS 2020, December 6-12, 2020, virtual*, 2020. URL https://proceedings.neurips.cc/paper/2020/hash/9d740bd0f36aaa312c8d504e28c42163-Abstract.html.

[10] Megan Tjandrasuwita, Jie Xu, Armando Solar-Lezama, and Wojciech Matusik. Memo: Meaningful, modular controllers via noise injection. In *Advances in Neural Information Processing Systems 37 (NeurIPS 2024)*, pages 18729–18757. Neural Information Processing Systems Foundation, 2024. doi: 10.52202/079017-0594. URL https://proceedings.neurips.cc/paper_files/paper/2024/hash/21912f7057935149fa58408ee8cb460e-Abstract-Conference.html.

[11] Jeevana Priya Inala, Osbert Bastani, Zenna Tavares, and Armando Solar-Lezama. Synthesizing programmatic policies that inductively generalize. In *Proceedings of the 8th International Conference on Learning Representations (ICLR)*, 2020. URL https://openreview.net/forum?id=S1l8oANFDH.